Collectives and Communication Kernels

Deep Dive into SGLang — Chapter 23

Yin Yang

Foundation Books Project

Where this chapter sits

This is the chapter that turns "move some bytes between GPUs" into "keep the model correct while it is split across ranks."

- Tensor-parallel serving makes one layer *look* local to each rank — but its output is a distributed object.
- A collective is what converts rank-local shards into the tensor the next operator may consume.
- In SGLang that conversion is a serving-correctness issue, not a background transport detail.

Goal: by the end you can look at a collective call and say who participates, which buffers are live, which backend path is legal, and how a wrong answer would be caught.

Roadmap

The layer boundary

Collective operations and cost

Memory and communicator state

Dispatch, kernels, and evidence

The layer boundary

Communication is a correctness boundary

Take the running two-rank row-parallel layer for `Llama-3.1-8B-Instruct`. Each rank multiplies by its weight shard:

$$\text{rank } 0: \ [1.0, 2.0] \qquad \text{rank } 1: \ [0.5, -1.0].$$

Neither rank holds the layer output. The sum all-reduce produces

$$[1.0, 2.0] + [0.5, -1.0] = [1.5, 1.0]$$

on *both* ranks, and only then may the next operator read the row.

Before the collective: a partial value. After a valid collective: the logical layer-boundary value. That visibility flip is the whole subject.

Participation alignment: the one thing every rank must share

The collective is valid only when all ranks agree on. . .
same ordered group, same operation, same tensor shape/dtype/layout, live registered buffers, the same fallback decision, and the same model-step boundary.

A CUDA graph adds one trap: replay reissues *recorded pointer arguments*, so a registered buffer must both keep its address and refresh its contents.

A launched collective is not usable. Usable means the boundary result is visible to the next operator, on every rank, for the same step.

Collective operations and cost

Three contracts over the same bytes

Operation	Rank-visible result	Book use
All-reduce	every rank gets $\bigoplus_j x_j$	row-parallel rejoin
All-gather	every rank gets concat$(x_0, \ldots)$	context-parallel rebuild
Reduce-scatter	rank j gets only its reduced shard	shard-owned outputs
All-to-all	each rank gets different routed rows	expert-parallel dispatch

Same movement of bytes, different rank-visible contract. A ring all-reduce is exactly reduce-scatter followed by all-gather — the fact the cost account below leans on.

The all-reduce cost account

Separate coordination from bytes moved:

$$T_{\text{allreduce}} \approx \underbrace{\alpha \log p}_{\text{coordination}} + \underbrace{2\beta n \frac{p-1}{p}}_{\text{movement}}.$$

For the canonical four-rank BF16 4096-element all-reduce: $p = 4$ and $n = 4096 \cdot 2 = 8192$ bytes per rank. A ring splits n into p chunks; each rank does $2(p-1)$ sends:

$$6 \cdot \frac{8192}{4} = 12{,}288 \text{ bytes} = \frac{2(4-1)}{4} \cdot 8192.$$

The formula is an account, not an algorithm. It is meaningful only after group order, p, n, dtype, and layout have fixed meaning.

Memory and communicator state

Link, transport, registered memory — kept apart

Three questions people collapse into "the network":

- **What path exists?** NVLink / PCIe P2P / RDMA fabric — a *link*.
- **Who may touch which bytes?** Registered memory $m = (a, n, d, \pi)$, valid over $\Lambda(m) = [t_{\text{reg}}, t_{\text{free}})$.
- **What does the step mean?** The *collective* defines group, order, reduction, and completion.

NVLink is an eligibility gate for a fast path — never the definition of all-reduce. Symmetric collective memory is registered memory rendezvoused across a group; it is not a synonym for RDMA.

From concept to code: the row-wise wait

SGLang's tensor-parallel helper makes "launched $\neq$ usable" explicit.

from `python/sglang/srt/layers/model_parallel.py`

```
class RowwiseParallelMaybeWait(RowwiseParallel):
    """
    A version of RowwiseParallel that waits for the output (establish dependency
    between comm stream and compute stream in CUDA sense) before going into the
    next op. This is needed to workaround the current interaction between
    AsyncCollectiveTensor and multi-platform ops, such as `RMSNorm`.
    """
    ...
    return torch.distributed._functional_collectives.wait_tensor(outputs)
```

The `wait_tensor` is the contract: RMSNorm may not read the row until the collective has actually completed.

Dispatch, kernels, and evidence

One logical call, an ordered ladder of backends

`GroupCoordinator.all_reduce` turns one request into a concrete path by an ordered eligibility check:

- Each row has a predicate $E_B(x, G, \tau, \rho, \mu)$ over tensor, group, topology, registration, and mode.
- The first enabled row whose predicate holds wins; a failed predicate does not skip the collective — it falls through.
- Graph capture changes which rows are legal (replay must reuse live captured addresses).

From concept to code: the eligibility gate

The custom fast path is guarded, not assumed.

from `python/sglang/srt/distributed/device_communicators/custom_all_reduce.py`

```
def should_custom_ar(self, inp: torch.Tensor):
    if self.disabled:
        return False
    inp_size = inp.numel() * inp.element_size()
    # custom allreduce requires input byte size to be multiples of 16
    if inp_size % 16 != 0:
        return False
    if not is_weak_contiguous(inp):
        return False
    if not _is_hip:
        if self.world_size == 2 or self.full_nvlink:
            return inp_size <= self.max_size
        return False
```

Byte alignment, weak contiguity, world size, NVLink, size gate — all eligibility, not performance tuning.

Correctness is tested with real ranks

- **Multi-rank test:** spawn $\{2, 4, 8\}$ processes, compare the custom result to a `torch.distributed` reference oracle with `assert_close`. A single process cannot expose a wrong rank order or bad IPC registration.
- **Determinism** is a *separate* claim: matching a reference once is not fixed-reduction-order under changing batch shapes.
- **Measurement**: name path eligibility *before* any speedup number. The reviewed slice is a portable `torch_nccl` fallback — payload scaling and overlap only, no custom-kernel claim.

We only make a claim as large as the metric, workload, and eligible path support.

What to carry forward

1. A collective is a **correctness boundary**: it flips a partial value into the logical layer output.
2. **Participation alignment** — ordered group, operation, tensor meaning, buffer lifetime, step boundary — is the master invariant.
3. The **cost account** $\alpha \log p + 2\beta n \frac{p-1}{p}$ is meaningful only after that metadata is fixed.
4. **Link** $\neq$ **transport** $\neq$ **collective**: NVLink and RDMA are eligibility, not meaning.
5. Dispatch is an **ordered fallback matrix**; every accepted row must deliver the *same* reduced tensor before the consumer runs.

Next: Section 1 opens the layer boundary that forces every rank to agree before the next operator can run.